Lab 7: Integració de YARA amb Wazuh

Detecció d'amenaces

GEI | FIB | Curs 2026/2027

UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat d'Informàtica de Barcelona



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat d'Informàtica de Barcelona

Detecció d'amenaces

Assignatura amb 22 hores de laboratori corresponent a l'especialitat:

Tecnologies de la informació

Inclosa en el:

Grau en Enginyeria Informàtica

Sobre mi

- **MANEL RODERO**
- Administrador de Sistemes, Universitat Politècnica de Catalunya:
  - UTGCNTIC [2019 – actualitat]
  - inLab FIB [2012 – 2019]
  - LCFIB [1996 – 2012]
- Analista de Seguretat, esCERT-UPC [2013 – 2017]
- Enginyer Informàtic, Facultat d'Informàtica de Barcelona [1999]
- Contacte:
  - manel.rodero@upc.edu

Lab 7. Integració de YARA amb Wazuh

Aplicar eines per centralitzar i correlacionar esdeveniments de seguretat

Peces necessàries

- **YARA**
  - Vist al Lab 6
  - Detecta patrons en fitxers mitjançant regles
- **Wazuh FIM**
  - Esmentat al Lab 5 (Integritat i monitorització de fitxers)
  - Detecta la creació, modificació o eliminació de fitxers
- **Wazuh Active Response**
  - Ho veurem avui ;-)
  - Executa accions automàtiques davant amenaces detectades

Què és una resposta activa?

- Mecanisme de Wazuh per **respondre automàticament** a una alerta, sense esperar la intervenció d'un humà
- S'executa a l'**Agent**, quan una regla concreta salta al Manager
  - Casos d'ús habituals: bloquejar una IP, aïllar un host, matar un procés, moure o esborrar un fitxer, etc.
- És la peça bàsica de l'orquestració i automatització de processos de resposta davant d'incidents de seguretat (SOAR) del Lab 10
- **L'objectiu d'avui**:
  - Muntar un pipeline **peça a peça i verificant-ne** cadascuna d'elles
  - Quan FIM detecti un fitxer nou, s'executarà YARA sobre ell

El pipeline d'avui en 3 fases

- **Detecció**
  - Un fitxer nou apareix a un directori vigilat per l'Agent
  - El FIM ho detecta i ...
  - ... el Manager rep l'alerta
- **Resposta automàtica**
  - L'Active Response executa resposta_yara.sh
  - Aquest script escaneja el nou fitxer amb regles YARA
- **Alerta final**
  - Si hi ha match de YARA, es genera un log que s'envia al Manager
  - Allà, una regla pròpia reconeix el missatge del log i ...
  - ... es genera una alerta al Dashboard

Les 4 peces que cal configurar

- **Agent**
  - FIM (<syscheck>)
    - Vigilar directoris en temps real
  - Script d'Active Response
    - Executa YARA quan salta el FIM
  - <localfile>
    - Envia el log de l'script de resposta automàtica al Manager
    - /var/ossec/logs/active-responses.log
- **Manager**
  - Decodificador personalitzat per al log
  - Regla personalitzada que reconeix el log i genera una alerta

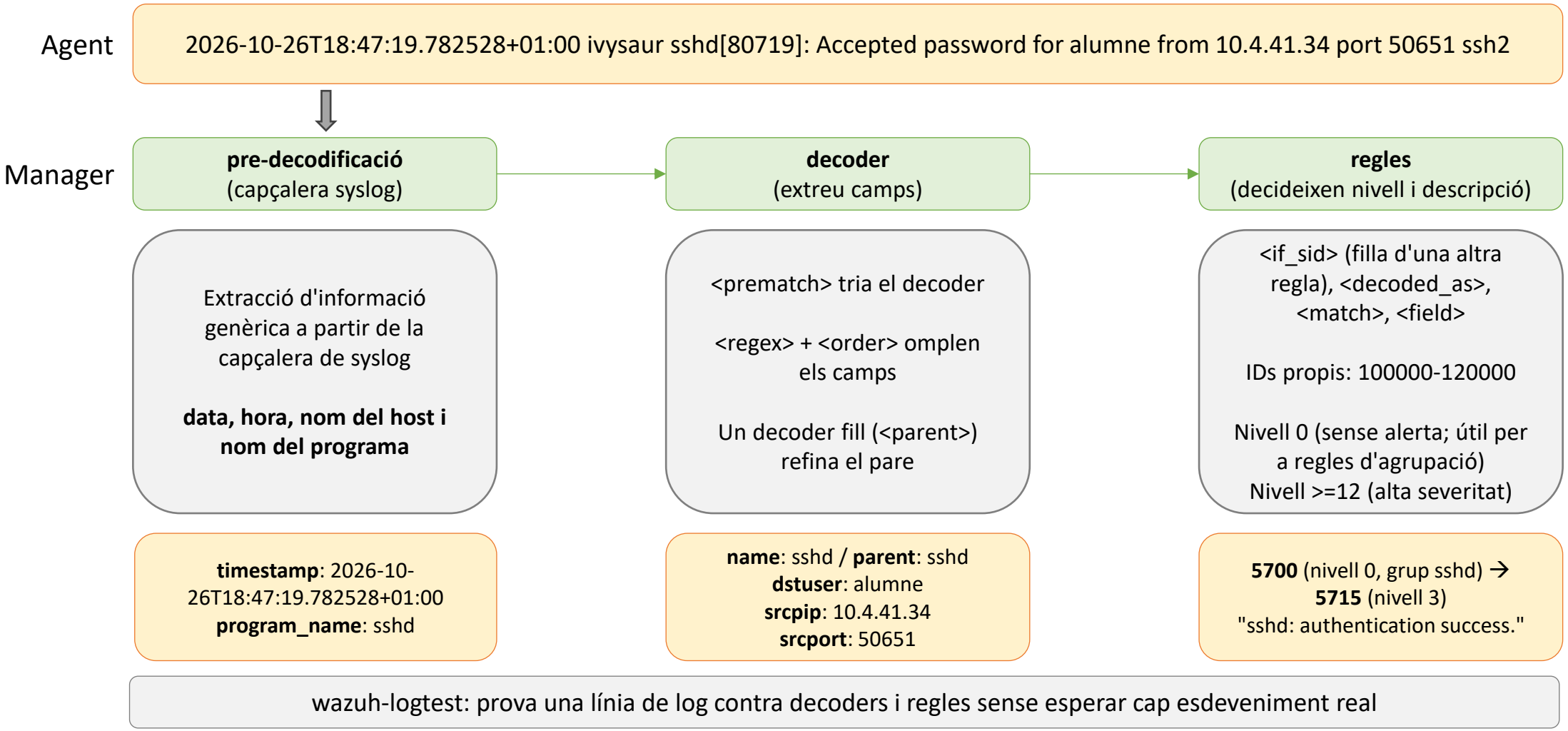
Com funciona el FIM?

- wazuh-syscheckd (agent) calcula una **línia base** per cada fitxer:
  - Hash, mida, permisos, propietari i mtime
- Modes de càlcul:
  - Escaneig periòdic: <frequency> (12h per defecte)
  - Temps real: realtime="yes", només directoris (inotify a Linux)
  - whodata="yes": a més, diu **qui** ha fet el canvi (auditd)
- El temps real comença quan acaba l'escaneig inicial
  - A ossec.log apareix "Real-time file integrity monitoring started"
- Fitxers que ja eren a la línia base → no generen "added"
- Regles del manager: **554** (afegit), **550** (modificat), **553** (eliminat)

Com funciona una resposta activa?

- Es declara al **Manager** amb dos blocs:
  - <command>: quin **executable** i amb quins <extra_args>
  - <active-response>: **quan** (rules_id, level, rules_group) i **on** (location)
    - location: local (l'agent que ha generat l'alerta), server, defined-agent, all
- L'script viu a **l'Agent**: /var/ossec/active-response/bin/
  - Propietari: root:wazuh (750). L'executa wazuh-execd com a root
- Rep l'alerta completa en JSON per STDIN:
  - command (add / delete), parameters.extra_args, parameters.alert (aquí hi ha syscheck.path)
- **Stateless** (una sola acció, timeout_allowed=no)
- **Stateful** (es desfà al cap d'un timeout, p.ex. desbloquejar una IP)

Del log a l'alerta: decoders i regles



wazuh-logtest

```
Starting wazuh-logtest v4.14.8
Type one log per line

2026-10-26T18:47:19.782528+01:00 ivysaur sshd[80719]: Accepted password for alumne from 10.4.41.34 port 50651 ssh2

**Phase 1: Completed pre-decoding.
       full event: '2026-10-26T18:47:19.782528+01:00 ivysaur sshd[80719]: Accepted password for alumne from 10.4.41.34 port 50651 ssh2'
       timestamp: '2026-10-26T18:47:19.782528+01:00'
       program_name: 'sshd'

**Phase 2: Completed decoding.
       name: 'sshd'
       parent: 'sshd'
       dstuser: 'alumne'
       srcip: '10.4.41.34'
       srcport: '50651'

**Phase 3: Completed filtering (rules).
       id: '5715'
       level: '3'
       description: 'sshd: authentication success.'
       groups: '['syslog', 'sshd', 'authentication_success']'
       firedtimes: '1'
       gdpr: '['IV_32.2']'
       gpg13: '['7.1', '7.2']'
       hipaa: '['164.312.b']'
       mail: 'False'
       mitre.id: '['T1078', 'T1021']'
       mitre.tactic: '['Defense Evasion', 'Persistence', 'Privilege Escalation', 'Initial Access', 'Lateral Movement']'
       mitre.technique: '['Valid Accounts', 'Remote Services']'
       nist_800_53: '['AU.14', 'AC.7']'
       pci_dss: '['10.2.5']'
       tsc: '['CC6.8', 'CC7.2', 'CC7.3']'
**Alert to be generated.
```

docker compose exec wazuh.manager /var/ossec/bin/wazuh-logtest

Font: https://documentation.wazuh.com/current/user-manual/reference/tools/wazuh-logtest.html

SSH Rules (0095-sshd_rules.xml)

```
<group name="syslog,sshd,">

  <rule id="5700" level="0" noalert="1">
    <decoded_as>sshd</decoded_as>
    <description>SSHD messages grouped.</description>
  </rule>

  <rule id="5715" level="3">
    <if_sid>5700</if_sid>
    <match>^Accepted|authenticated.$</match>
    <description>sshd: authentication success.</description>
    <mitre>
      <id>T1078</id>
      <id>T1021</id>
    </mitre>
    <group>authentication_success</group>
  </rule>

</group>
```

Font: https://github.com/wazuh/wazuh-ruleset/blob/master/rules/0095-sshd_rules.xml

Exercicis dirigits



- Abans de començar, cal **clonar** o **actualitzar** el repo local
- Cada exercici finalitza amb una **comprovació** del resultat esperat
- No es passa al següent exercici fins que la comprovació sigui correcta
- On s'executen les comandes?
  - [A] = màquina amb l'Agent
  - [M] = màquina amb el Manager (directori single-node)

D0 - Preparació i comprovació de l'entorn

- [A] Instal·lar **jq**, un processador de JSON lleuger, flexible i de CLI
  - Útil per obtenir les alertes sense accedir al Dashboard
- [A] Comprovar que YARA està instal·lat i la seva versió
- [A] Comprovar que l'Agent de Wazuh està en execució
- [A] Comprovar que l'Agent envia logs d'Active Responses al Manager
- [A] Crear els directoris "watch" i "rules" per aquest Lab
- [M] Comprovar que l'Agent està actiu al Manager

D1 - FIM sol, sense YARA

- [A] Afegir el directori "watch" al <syscheck> de l'Agent
- [A] Reiniciar l'Agent i comprovar que s'activa el FIM en temps real

YYYY/MM/DD HH:MM:SS wazuh-syscheckd: INFO: (6012): Real-time file integrity monitoring started.

- [M] Afegir regles filles del FIM al fitxer de regles locals del Manager
  - Editar el fitxer del contenidor o el fitxer local si s'ha mapejat el volum
- [M] Reiniciar el contenidor
- [A] Crear un fitxer al directori vigilat "watch"
- **Comprovar** que l'alerta apareix al Dashboard (Threat Hunting)
  - Veure tots els esdeveniments o filtrar-los
  - Hauria d'aparèixer 1 esdeveniment (rule.id = 100301)
  - Inspeccionar els detalls del document

D2 - Decoder i regles YARA

- **Objectiu**: que el Manager entengui les respostes de YARA i les converteixi en alertes
- [M] (local_decoder.xml) Afegir un decoder que reconegui les respostes de YARA i n'extregui els camps que es cregui convenient
- [M] (local_rules.xml) Afegir regles per agrupar i alertar si és necessari
- [M] Comprovar la sintaxi dels fitxers XML amb wazuh-analysisd -t
- [M] Provar una línia d'exemple (sense esperar cap esdeveniment real) amb wazuh-logtest
- [M] Reiniciar el Manager perquè wazuh-analysisd carregui el decoder i les noves regles

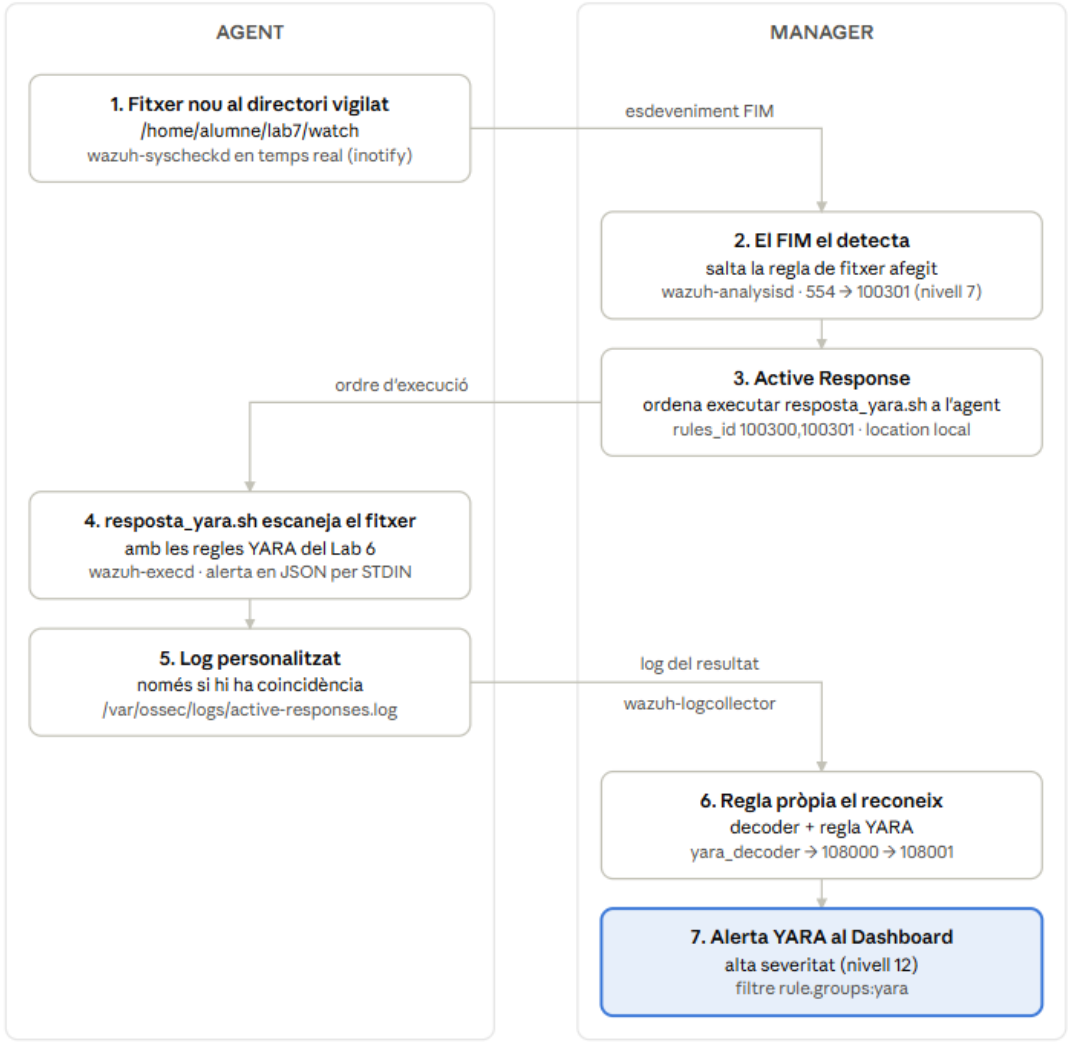
D3 - Regla YARA i script

- [A] Crear regla a ~/lab7/rules/eicar_rule.yar per detectar EICAR
- [A] Crear un índex de regles a ~/lab7/rules/index.yar
- [A] Provar les regles amb un fitxer eicar.com **fora del directori vigilat**
- [A] Crear script de resposta activa resposta_yara.sh
- [A] Simular comportament de wazuh-execd (JSON mínim per STDIN)
- [A] Comprovar el log de respostes actives
- **Comprovar** que apareix al Dashboard (Threat Hunting)
  - Veure tots els esdeveniments o filtrar-los
  - Hauria d'aparèixer 1 esdeveniment (rule.id = 108001)
  - Inspeccionar els detalls del document

D4 - Connectar la resposta activa

- [M] Modificar <ossec_config> al fitxer ossec.conf del contenidor
  - Afegir una comanda "yara_linux" per executar resposta_yara.sh
  - Afegir una resposta activa per executar la comanda anterior
- [M] Reiniciar el contenidor i comprovar que la configuració persisteix
- [A] Reiniciar l'Agent per seguretat
- [A] **Provar la resposta** activa copiant eicar.com al **directori vigilat**
- [A] Comprovar que apareix al log de respostes actives
- **Comprovar** que apareix al Dashboard (Threat Hunting)
  - Veure tots els esdeveniments o filtrar-los
  - Hauria d'aparèixer 1 esdeveniment de nivell 7 (rule.id = 100301)
  - Hauria d'aparèixer 1 esdeveniment de nivell 12 (rule.id = 108001)

El pipeline dels exercicis dirigits...



Treball autònom en parelles



- El treball s'ha d'entregar en **1 únic fitxer PDF per grup** amb:
  - Contingut del fitxer *.yar
  - Comandes executades i resultat
  - Taules i/o fitxes de resultats
- El nom del fitxer ha de tenir aquest format:
  - `Lab7_grupXX_Cognom1_Cognom2.pdf`
- L'entrega es fa mitjançant el **Racó**
- Teniu fins a la **mitjanit** del següent dimarts o divendres per entregar

A1. La vostra regla del Lab 6 (1 punt)

- Copiar reverse_shell_v1.yar del Lab 6 al directori de regles del Lab 7
- Incloure el nou fitxer a l'índex de regles YARA creat al D3
- Provar les regles d'aquest índex amb un fitxer fora del directori vigilat
  - P. ex. el fitxer ~/lab6/mostres/sample_suspicious1.txt
- Copiar el fitxer anterior al directori vigilat "watch"

- *Apareixen noves línies al fitxer active-responses.log? Quines?*
- *Apareixen noves alertes al Dashboard? Quines?*
- *Recollir les alertes des del fitxer alerts.json mitjançant la comanda jq*

A2. Control amb fitxer net (1 punt)

- Copiar un fitxer net al directori vigilat "watch"
  - P. ex. el fitxer ~/lab6/mostres/sample_clean.txt

- *Apareixen noves línies al fitxer active-responses.log? Quines?*
- *S'ha executat la resposta activa? Qui l'ha disparada?*
- *Apareixen noves alertes al Dashboard? Quines?*
- *Recollir les alertes des del fitxer alerts.json mitjançant la comanda jq*

A3. Experiments amb predicció (4 punts)

- Per a cada experiment:
  - Escriure **primer** a la fitxa quines alertes (Id) s'esperen veure
  - Després executar l'acció i anotar les alertes observades
- Es corregeixen les observades; la predicció només es valora si la justificació de la diferència és correcta
- Experiments:
  - E1: Afegir contingut sospitós a contingut net
  - E2: Copiar eicar.com al directori /tmp
  - E3: Copiar eicar.com a un subdirectori del directori vigilat
  - E4: Esborrar el fitxer eicar.com del directori vigilat

A4. Quarentena automàtica (opcional)

- Modificar l'script de resposta YARA perquè si hi ha coincidència:
  - **Mogui** el fitxer a un directori de quarantena fora del directori vigilat
  - **Registri** aquest moviment al fitxer de log de respostes actives
    - wazuh-yara: INFO - Quarantined: $FILENAME
  - **Pista**: Cal modificar el bloc final de l'script
- Crear una nova regla al Manager per generar l'alerta del moviment
  - Regla amb identificador **108002**, nivell **10** i descripció:
    - Fitxer "$(yara_scanned_file)" mogut al directori de quarantena
  - Decoder per extreure els camps necessaris del missatge
  - **Pista**: Cal modificar local_rules.xml i local_decoder.xml al Manager
- Copiar el fitxer eicar.com al directori vigilat
- *Quantes alertes es generen? Quines?*

Aquesta és la primera resposta automàtica real: el pas cap al **SOAR** del Lab 10

Reflexions #1

- **Quina diferència hi ha entre l'alerta 100301 i la 108001?**
  - 100301 salta per a qualsevol fitxer nou
    - Depèn només de l'esdeveniment al sistema de fitxers, no del contingut
  - 108001 salta només si el contingut coincideix amb una regla YARA
    - El FIM és el disparador i YARA el classificador
- **Per què cal que el resultat passi per un fitxer de log recollit per l'agent?**
  - L'script s'executa a l'agent i aquest no pot crear alertes
    - Només wazuh-analysisd al Manager genera alertes a partir d'esdeveniments
  - El fitxer active-responses.log és llegit per wazuh-logcollector que envia cada línia com un esdeveniment que es decodifica i passa per les regles
    - El resultat queda centralitzat, indexat i correlacionat amb la resta d'esdeveniments de l'agent

Reflexions #2

- **Quins són els riscos de vigilar la carpeta de descàrregues d'un usuari real?**
  - Falsos positius de regles genèriques
  - Cost d'escanejar cada descàrrega i fitxers grans
  - Fitxers a mig descarregar (.part, .crdownload) que generen molts "modificat" i escanejos repetits
  - Arxius comprimits o xifrats que YARA no inspecciona per dins
  - El fitxer es pot obrir abans que acabi l'escaneig
  - Privacitat del contingut de l'usuari
  - Manteniment de les regles
  - Etc.